

Advised by: Professor Ming Chow & Professor Steven Bell
April 22, 2025

Design Overview

Multi-Layer Cryptographic Defense

- Inner Layer: AES-GCM**
 - Each channel uses a unique key (derived from HKDF) to encrypt the frame payload with AES-GCM ensuring confidentiality and integrity via authentication tags and random nonces.
- Outer Layer: AES-CBC + HMAC-SHA256**
 - The inner ciphertext and metadata (channel no, timestamp) are encrypted with AES-CBC using a system-wide master key. An HMAC-SHA256 tag verifies integrity before decryption and prevents tampering and replay attacks.

Working Together: Defense In Depth

Even if one layer is bypassed, the other still remains a barrier.

- If AES-CBC is decrypted, the attacker still needs the per-channel AES-GCM key.
- If AES-GCM fails, the HMAC and timestamp checks in the outer layer will catch the issue.

This dual-layer design ensures:

- Confidentiality of both data and metadata
- Integrity via GCM tag and HMAC
- Authentication & Access Control via subscription validation and channel keys
- Replay protection via encrypted, increasing timestamps.

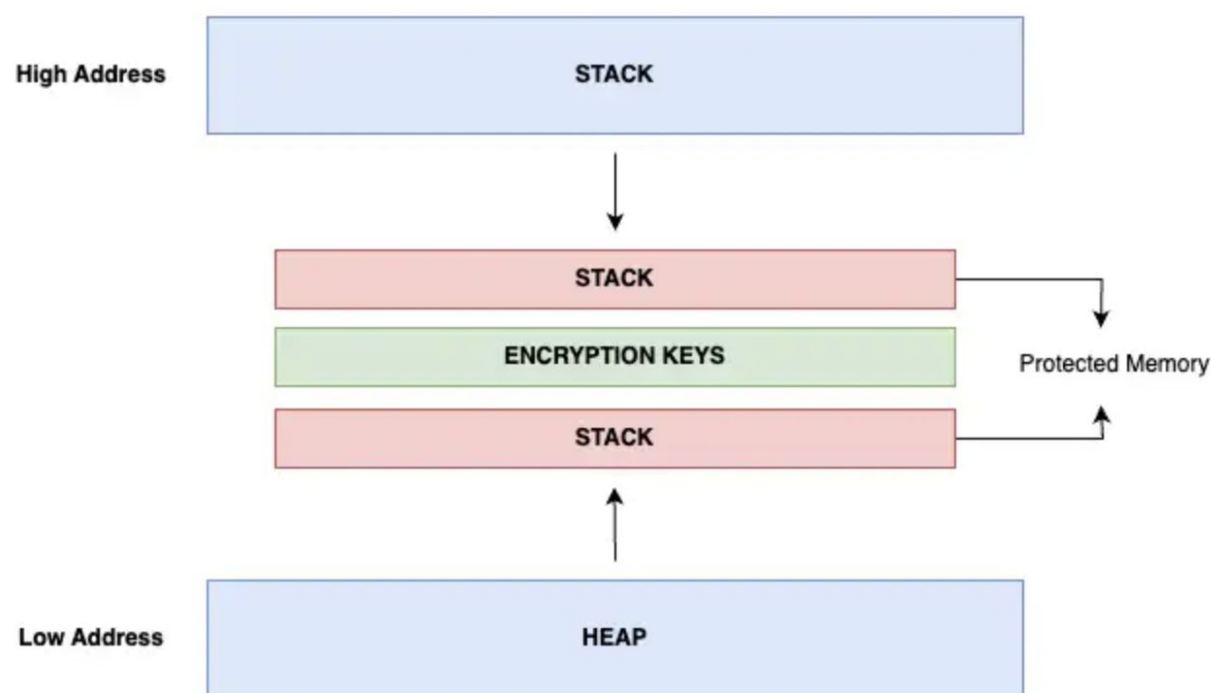
Memory Storage & Protection

Securing encryption keys is critical to preventing unauthorized access and exploitation of sensitive data. The Memory Protection Unit (MPU) allows us to enforce access control policies on memory regions, reducing the risk of key leakage, buffer overflows, and other attacks.

Our primary strategy involved the use of guard regions (privileged memory areas) to protect buffer overflow from the stack or the heap into the region where the encryption keys are stored. No supervisor call is needed to retrieve the keys because the keys themselves are not stored in a protected region.

Key Security Concerns & Mitigations For Memory Protection

- Buffer Overflow:** Establish MPU guard regions that limit access to key storage and detect unauthorized overflows reducing the risk of memory corruption and key leakage.
- Stack & Heap Overflow:** Restrict access between the heap and the stack and utilize memory allocators to prevent large, unchecked memory allocations.
- Code Injection Attacks:** Prevent execution in key and data storage areas.



Defensive Highlight

Replay Attack Mitigation

Replay attacks are a common threat in embedded systems, specifically in broadcast architecture like satellite TV. In our specific case, we were concerned about an attacker intercepting a previously valid frame and re-sending it to gain access.

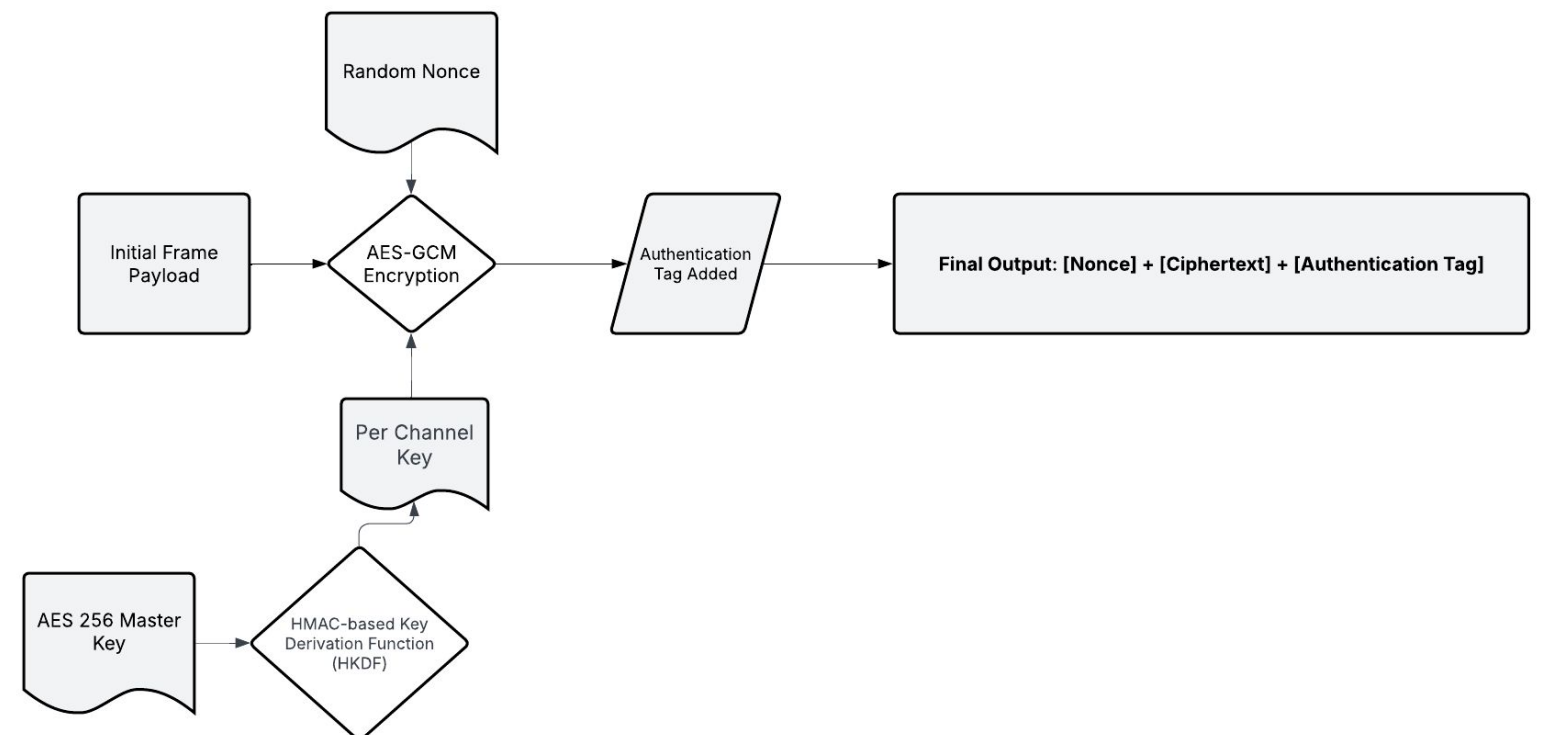
The Solution:

Encrypted timestamps were compared (after decryption of the metadata) against the last accepted timestamp to reject reused or stale frames. This design aimed to prevent attackers from exploiting expired or reused tokens. This worked on all channels, except for one where we did not implement it.

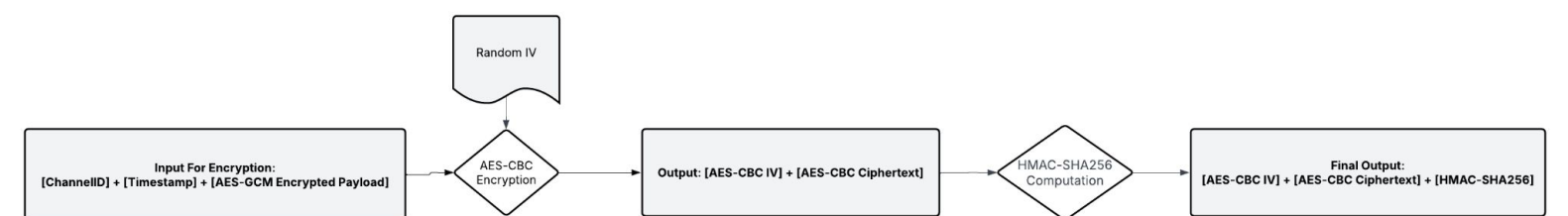
Where It Didn't Work

In our effort to leave Channel 0 as an open and public emergency broadcast, we decoded all frames from channel 0, without ever checking the timestamps. We didn't think of the security ramifications that could come with that. In the future, we would work to ensure that Channel 0 is still digitally signed with a private key. Decoders can then verify the signature using a hardcoded public key, while still preventing spoofed or tampered messages.

Inner Layer



Outer Layer



Offensive Highlight

As referenced in our defensive highlight, we implemented monotonically increasing timestamp checks per frames in every channel (except channel 0) to prevent replay attacks. Once we realized what we had done on our own design, we wondered if other people had made the same mistake and if we could exploit that to achieve the Pesky Neighbor flag.

How to break in via Channel 0:

- The attacker (us in this case) captures the full frame, including unencrypted timestamp, metadata, and payload (timestamps are unencrypted because of channel 0).
- The decoder does not check the timestamp on channel 0 allowing the remainder of the attack to happen.
- The attacker forwards the same frame using decoder.py and the decoder accepts it as valid.
- Because the decoder believes it's receiving legitimate Channel 0 data, the flag is captured.

To do this, we looked for similar instances in the decoder to the mistake we made:

```
// Timestamp Validation
timestamp_status_t verify_frame_timestamp(channel_id_t channel, timestamp_t frame_timestamp)
{
    // Emergency channel bypass
    if (channel == EMERGENCY_CHANNEL)
    {
        return TIMESTAMP_VALID;
    }
}
```

What this does: Any frame with channel == 0 is treated as having a valid timestamp even if it's replayed, out of range, or completely forged.

Future Iterations: Unfortunately, we uncovered this fatal flaw rather late in the attack phase of the competition. However, automating the detection of this critical error would have allowed for easy exploitation of a key vulnerability on several teams.

